

INTERNATIONAL UNIVERSITY
IT096IIU – NETCENTRIC
**LAB 01 – GO FOUNDATION
FOOD ORDERING SYSTEM**

Practical Laboratory | Group of 2 | 150 minutes | 100 points

Core principle: Think together. Build with AI. Verify together. Own the result.

Lab Flow

THINK → **BUILD** → **VERIFY** → **ADAPT** → **GROUP CHECK**
AI OFF AI ON AI ON AI ON OWN IT

1 Scenario

You are developing a small **Food Ordering System** for a campus food shop. The system keeps a menu of food items and allows a customer to create an order.

Each food item has: **Name**, **Price**, **Category**, and **Available**.

Initial categories: Main Dish, Drink, Dessert.

The system must support:

1. Display available food.
2. Search food by name.
3. Add food to an order.
4. Reject invalid quantities.
5. Reject unavailable food.
6. Calculate the order subtotal.
7. Display an order summary.
8. Apply a discount when the requirement changes.

2 THINK – 20 minutes, AI OFF

Work as a pair. **Do not use AI during this phase.**

Discuss and write short answers to at least these questions:

1. What information belongs to a **FoodItem**?
2. What information belongs to an **Order**?
3. Can an order contain several different food items?
4. How should quantity be represented?
5. Where should food items be stored?
6. Where could a **map** be useful?
7. What should happen when requested food does not exist?
8. What should happen when quantity is 0 or negative?
9. What should happen when a food item is unavailable?
10. Which functions/methods should be responsible for business rules?

Create a short design containing:

- **FoodItem**

- **Order**
- at least one slice and one map
- important functions/methods
- two possible designs
- the chosen design and one reason
- four predicted failure cases

Before coding, show your design to the instructor. The instructor may clarify requirements or concepts, but will not provide the complete solution.

3 BUILD – 60 minutes, AI ON

You may use AI tools. **AI may generate code, but your pair must understand and own the code.**

3.1 Example: FoodItem

```
type FoodItem struct {  
    Name      string  
    Price     float64  
    Category  string  
    Available bool  
}
```

This is an example, not the required final design.

3.2 Example: slice

```
menu := []FoodItem{  
    {  
        Name:      "Chicken Rice",  
        Price:     45000,  
        Category:  "Main Dish",  
        Available: true,  
    },  
    {  
        Name:      "Milk Tea",  
        Price:     25000,  
        Category:  "Drink",  
        Available: true,  
    },  
    {  
        Name:      "Cheesecake",  
        Price:     35000,  
        Category:  "Dessert",  
        Available: false,  
    },  
}
```

Discuss why a slice is suitable, how an item would be found, and what should happen when it is unavailable.

3.3 Example: map lookup

```
menuByName := map[string]FoodItem{  
    "Chicken Rice": {  
        Name:      "Chicken Rice",
```

```
        Price:    45000,
        Category: "Main Dish",
        Available: true,
    },
    "Milk Tea": {
        Name:      "Milk Tea",
        Price:     25000,
        Category:  "Drink",
        Available: true,
    },
}

food, ok := menuByName["Milk Tea"]

if !ok {
    fmt.Println("Food not found")
} else {
    fmt.Println(food.Name, food.Price)
}
```

Do not simply copy the example. Decide whether the final design should use a slice, a map, or both.

3.4 Example: subtotal function

```
func CalculateSubtotal(items []FoodItem, quantities []int) float64 {
    total := 0.0

    for i := range items {
        total += items[i].Price * float64(quantities[i])
    }

    return total
}
```

Discuss the assumptions: different slice lengths, negative quantities, where validation belongs, and whether another data structure is safer.

3.5 Example: validation function

```
func IsValidQuantity(quantity int) bool {
    return quantity > 0
}
```

Your implementation must contain:

- at least 6 food items and 3 categories
- display available food and search by name
- add an item, validate quantity, reject unavailable food
- subtotal and order summary
- at least one slice and one map
- functions and/or methods
- optional CLI menu

3.6 AI collaboration log

Record at least 3 meaningful AI interactions, including at least one review/verification request. Do not submit a huge transcript.

#	What we asked AI	What AI suggested	What we changed/verified
1			
2			
3			

4 VERIFY – 25 minutes

Test the program using at least:

Case	Expected behavior	Result
Normal order	Correct subtotal	
Quantity = 1	Accepted	
Quantity = 0	Rejected	
Negative quantity	Rejected	
Unavailable food	Rejected	
Food not found	Handled safely	
Several different items	Correct total	

Intentionally introduce one small defect, for example:

```
total += items[i].Price
```

instead of:

```
total += items[i].Price * float64(quantities[i])
```

Then identify, explain, fix, and retest the defect.

Ask AI to review one important function without rewriting it. Compare the AI review with your own reasoning.

5 ADAPT – 15 minutes

New requirement:

Orders with a subtotal of at least **300,000 VND** receive a **10% discount**.

Display:

Subtotal: 350000

Discount: 35000

Final Total: 315000

Test:

1. Below 300,000 VND
2. Exactly 300,000 VND
3. Above 300,000 VND

Reflect in 2–3 sentences: Was the original design easy or difficult to change? Which part would become difficult if several discount rules were introduced?

6 GROUP CHECK – 15 minutes

The instructor checks the **pair as a group**. Both members should be able to explain the solution.

Possible challenges:

- Explain your struct.
- Explain why you used a slice/map.
- Explain one important function.
- Find an edge case.
- Fix a small bug.
- Add search by category.
- Explain an AI-generated part.
- Change the discount rule.

The purpose is not to catch AI use. The purpose is to confirm that the pair owns the solution.

The instructor may ask either member one short follow-up question if clarification is needed.

7 Submission

Submit:

1. Source code
2. Test cases/results
3. Short design note
4. AI collaboration log
5. Short ADAPT reflection

Both students are responsible for the final submission.

8 Assessment

Evidence	Weight
THINK	20
BUILD	20
VERIFY	20
ADAPT	15
GROUP CHECK	25
Total	100

9 Homework – One-week extension

Group of 2; maximum 3 hours per group.

Add customer types:

- Regular
- Member – 5% discount on food subtotal

Add order types:

- Pickup – 0 VND
- Delivery – 30,000 VND

Use:

`Final Total = Subtotal - Customer Discount + Delivery Fee`

New clarification:

The member discount applies only to the food subtotal. It does not reduce the delivery fee.

Submit updated source code, tests, short design note, one AI review, and a short explanation of how the requirement change was handled. Both members must understand the final solution.

Final Principle

Think together. Build with AI. Verify together. Own the result.